

SLIIT Year 1 Semester 1 Mid Examination Practice

C Programming Error Analysis

Focus: Syntax Errors, Compile-Time Errors, and Run-Time Errors

University: Sri Lanka Institute of Information Technology (SLIIT)

Degree: B.Sc. (Hons) in Computer Science

Module: SE1012 - Programming Methodology

Academic Level: Year 1, Semester 1

Assessment: Mid Examination Practice

Question Type: 40 Multiple Choice Questions

Suggested Duration: 60 minutes

Total Marks: 40

Based exclusively on the uploaded teaching material

Lab 2-Lab 6, Lecture 1 Parts A and B, Lecture 2, Lecture 3, Lesson 4, and Lesson 5.

Instructions

1. Answer all 40 questions.
2. Select the single best answer for each question.
3. Each question carries 1 mark.
4. Assume a conforming C compiler with normal diagnostic messages enabled.
5. Unless stated otherwise, assume all required variables have the types shown in the code.
6. Distinguish carefully among:
 - **syntax errors**, where the program violates C grammar;
 - **other compile-time errors**, detected while translating the source;
 - **link-time errors**, detected while creating the executable; and
 - **run-time or logical errors**, which occur after a program has been built.
7. Code indentation is intentional, but remember that C uses braces rather than indentation to group statements.

Difficulty blueprint

- Easy: 12 questions (30%)
- Medium: 20 questions (50%)
- Hard: 8 questions (20%)

The questions progress from foundational identification to mixed debugging and code analysis.

Multiple Choice Questions

Questions 1-12

1.

What is the most direct result of omitting the semicolon after the following statement?

```
int total = 25
```

- A. A run-time division error
- B. A syntax error reported during compilation
- C. A link-time error
- D. The value of total becomes zero

2.

Which declaration contains an invalid C variable name?

- A. `int total_marks;`
- B. `double _average;`
- C. `int 2marks;`
- D. `char section1;`

3.

Consider a source file that calls `printf` but does not include `<stdio.h>`. What is the best diagnosis under modern C compilation?

- A. The preprocessor automatically adds `<stdio.h>`
- B. The compiler issues a diagnostic because the function declaration is unavailable
- C. The program must produce a division-by-zero error
- D. `printf` is treated as a comment

4.

Which Linux command compiles `check.c` and requests an executable named `check`?

- A. `gcc check -o check.c`
- B. `./check.c check`
- C. `gcc check.c -o check`
- D. `gcc -run check.c`

5.

Which stage combines object modules and required library routines to create an executable?

- A. Editing
- B. Linking
- C. Input validation
- D. Looping

6.

Which statement correctly reads an integer into `age`?

- A. `scanf("%d", age);`
- B. `scanf("%f", &age);`
- C. `scanf("%d", &age);`
- D. `scanf("age", %d);`

7.

Which operator must replace the blank to compare `score` with 75?

```
if (score ____ 75)
```

- A. `=`
- B. `==`

- C. !=
- D. >=

8.

The following program is compiled and then the user enters 0. What is the principal problem?

```
int divisor;
scanf("%d", &divisor);
printf("%d\n", 100 / divisor);
```

- A. Missing semicolon
- B. Invalid identifier
- C. Division by zero during execution
- D. Missing **break**

9.

Which statement is essential if the following loop must eventually terminate normally?

```
int i = 1;
while (i <= 5) {
    printf("%d ", i);
    /* missing statement */
}
```

- A. i++;
- B. i = 1;
- C. **continue**;
- D. printf("%d", 5);

10.

What is the usual purpose of **break** at the end of a **switch** case?

- A. To repeat the same case
- B. To prevent unintended fall-through into later cases
- C. To declare the switch variable
- D. To restart main

11.

What does a function prototype placed before main provide?

- A. A loop termination condition
- B. The function's name, return type, and parameter types before it is called
- C. The function's final output only
- D. An automatic value for every parameter

12.

Which is the correct **do-while** structure?

A.

```
do {  
    work();  
} while (condition);
```

B.

```
do (condition) {  
    work();  
}
```

C.

```
while do (condition) {  
    work();  
}
```

D.

```
do {  
    work();  
} while condition
```

Questions 13-32

13.

What does this program print?

```
int pieces = 5 / 2;  
printf("%d\n", pieces);
```

- A. 2
- B. 2.5
- C. 3
- D. It cannot compile

14.

What is printed by the following code?

```
int score = 0;  
  
if (score = 75)  
    printf("Pass\n");  
else  
    printf("Fail\n");
```

- A. Fail
- B. Pass
- C. Nothing
- D. A mandatory syntax error occurs

15.

Which condition correctly accepts water only when it is strictly between 0.1 and 1.5?

- A. `water > 0.1 || water < 1.5`
- B. `0.1 < water < 1.5`
- C. `water > 0.1 && water < 1.5`
- D. `water != 0.1 || water != 1.5`

16.

Why is the following condition safe when `x` is zero?

```
if (x != 0 && 20 / x > 2)
    printf("High\n");
```

- A. Integer division automatically changes zero to one
- B. `&&` short-circuits, so `20 / x` is not evaluated
- C. `printf` prevents the division
- D. Division by zero is legal inside an `if`

17.

Given `a = 6`, `flag = 0`, and `c = 4`, what is the value of this expression?

```
a != 7 && flag || c >= 6
```

- A. 0
- B. 1
- C. 4
- D. A syntax error

18.

What happens when this code runs?

```
int i = 1;

while (i <= 3); {
    printf("%d ", i);
    i++;
}
```

- A. It prints 1 2 3
- B. It prints 1 once
- C. It remains in an empty infinite loop
- D. The semicolon causes a compulsory link error

19.

Why does this loop become stuck when `i` reaches 3?

```
int i = 1;

while (i <= 5) {
    if (i == 3)
        continue;
    printf("%d ", i);
    i++;
}
```

- A. `continue` exits the loop permanently
- B. `continue` skips `i++`, so `i` remains 3
- C. `printf` changes `i` to 3
- D. A `while` loop cannot contain `continue`

20.

What is the exact output?

```
int option = 2;

switch (option) {
    case 1:
        printf("A");
        break;
    case 2:
        printf("B");
    case 3:
        printf("C");
        break;
    default:
        printf("D");
}
```

- A. B
- B. BC
- C. BCD
- D. D

21.

What is the most accurate statement about this call?

```
printf("%d\n", 5.5);
```

- A. It is guaranteed to print 5
- B. It is guaranteed to print 5.500000
- C. The format and argument types do not match, so no dependable output is guaranteed
- D. %d automatically converts every **double** safely

22.

Which statement correctly reads a value into **double** distance?

- A. scanf("%f", distance);
- B. scanf("%lf", &distance);
- C. scanf("%d", &distance);
- D. scanf("%lf", distance);

23.

What is the compiler's main objection to this expression?

```
double a = 7.5, b = 2.0;
double r = a % b;
```

- A. % requires integer operands
- B. % requires a **char** result
- C. % can be used only inside a loop
- D. The expression is completely correct

24.

A program calls sqrt without including <math.h>. Which correction addresses the missing declaration?

- A. Add **#include** <math.h>
- B. Add **break;** before sqrt
- C. Replace sqrt with scanf
- D. Add a semicolon after the **#include** line

25.

What is wrong with these declarations in the same program?

```
double calculate(double x);

int calculate(double x) {
    return x * x;
}
```

- A. The prototype and definition have conflicting return types
- B. A function cannot accept a **double**
- C. Every function must return **void**
- D. The parameter must be named differently in the definition

26.

What is printed?

```
void change(int n) {
    n = n + 10;
}

int main(void) {
    int value = 5;
    change(value);
    printf("%d\n", value);
    return 0;
}
```

- A. 5
- B. 10
- C. 15
- D. A syntax error

27.

What is the danger in this code?

```
char word[5] = {'S', 'L', 'I', 'I', 'T'};
printf("%s\n", word);
```

- A. %s requires an integer
- B. The array has no null terminator, so printing it as a string has undefined run-time behavior
- C. Character constants must use double quotes
- D. Arrays cannot contain five elements

28.

What is printed by this correctly declared function sequence?

```
void second(void);

void first(void) {
    printf("A");
    second();
}

void second(void) {
    printf("B");
}

int main(void) {
    first();
    return 0;
}
```

- A. A

- B. B
- C. AB
- D. BA

29.

The intention is to print 1 through 5. What is the logical error?

```
int i;  
for (i = 1; i < 5; i++)  
    printf("%d ", i);
```

- A. Initialization is missing
- B. The condition should be $i \leq 5$
- C. The update should be $i--$
- D. A **for** loop must always contain **break**

30.

A loop fills reversed with all characters of word, but the program later prints reversed using %s. Which statement is required after the loop?

```
reversed[length] = ____;
```

- A. `'\0'`
- B. `'\n'`
- C. `0.1`
- D. `"%s"`

31.

What does this program fragment print?

```
int i;  
  
for (i = 0; i < 3; i++)  
    printf("%d ", i);  
    printf("Done");
```

- A. 0 Done 1 Done 2 Done
- B. 0 1 2 Done
- C. Done
- D. It fails because indentation controls the loop body

32.

What happens at compilation when the program attempts the following assignment?

```
const int MAX = 100;  
MAX = 120;
```

- A. MAX changes to 120
- B. MAX changes only at run time
- C. A diagnostic is required because a const-qualified object is not modifiable
- D. The assignment is ignored as whitespace

Questions 33-40

33.

A file contains valid helper-function definitions but no main function. The compiler creates object code successfully. What normally happens when an executable is requested?

- A. The preprocessor invents main
- B. The linker reports that the required program entry point is missing
- C. The first helper function automatically becomes main
- D. The executable starts from the final helper function

34.

What is printed by the following code?

```
int i, j;

for (i = 1, j = 9; i < j; i += 2, j -= 2);
printf("%d %d\n", i, j);
```

- A. 1 9
- B. 3 7
- C. 5 5
- D. It is an infinite loop

35.

What is printed when d is zero?

```
int d = 0;
int x = 12;

if (d == 0 || x % d == 0)
    printf("Protected\n");
else
    printf("Other\n");
```

- A. Protected
- B. Other
- C. Nothing, because $x \% d$ must execute first
- D. A mandatory syntax error

36.

Which is the only correctly matched input/output pair for **double** value?

A.

```
scanf("%f", value);
printf("%d\n", value);
```

B.

```
scanf("%lf", &value);
printf("%.2f\n", value);
```

C.

```
scanf("%d", &value);
printf("%lf\n", &value);
```

D.

```
scanf("%lf", value);  
printf("%c\n", value);
```

37.

The programmer expects `x` to become 9. What is the actual result and best diagnosis?

```
void adjust(int value) {  
    value = value + 5;  
}  
  
int main(void) {  
    int x = 4;  
    adjust(x);  
    printf("%d\n", x);  
    return 0;  
}
```

- A. It prints 9; there is no error
- B. It prints 4; pass-by-value changes only the function's local copy
- C. It prints 5; the argument is discarded
- D. It cannot compile because functions cannot receive integers

38.

What is the output?

```
int i = 0, j, total = 0;  
  
do {  
    j = 0;  
    do {  
        total++;  
        j++;  
    } while (j < 2);  
    i++;  
} while (i < 3);  
  
printf("%d\n", total);
```

- A. 2
- B. 3
- C. 5
- D. 6

39.

What is the exact output?

```
int i = 0;

while (i < 8) {
    i++;

    if (i % 3 == 0)
        continue;

    if (i == 7)
        break;

    printf("%d ", i);
}
```

- A. 1 2 4 5
- B. 1 2 3 4 5 6 7
- C. 1 2 4 5 7 8
- D. The loop is infinite

40.

Which control structure safely avoids both division by zero and taking the square root of a negative discriminant in a real-root quadratic solver?

A.

```
root = (-b + sqrt(d)) / (2.0 * a);  
if (a == 0.0 || d < 0.0)  
    printf("Invalid\n");
```

B.

```
if (a == 0.0) {  
    printf("Not quadratic\n");  
} else {  
    d = b * b - 4.0 * a * c;  
    if (d < 0.0)  
        printf("No real roots\n");  
    else  
        root = (-b + sqrt(d)) / (2.0 * a);  
}
```

C.

```
d = b * b - 4.0 * a * c;  
if (d <= 0.0)  
    printf("No roots\n");  
else  
    root = (-b + sqrt(d)) / 0.0;
```

D.

```
d = b * b - 4.0 * a * c;  
if (a != 0.0 || d >= 0.0)  
    root = (-b + sqrt(d)) / (2.0 * a);
```

Answer Key

Question	Answer	Question	Answer	Question	Answer	Question	Answer
1	B	11	B	21	C	31	B
2	C	12	A	22	B	32	C
3	B	13	A	23	A	33	B
4	C	14	B	24	A	34	C
5	B	15	C	25	A	35	A
6	C	16	B	26	A	36	B
7	B	17	A	27	B	37	B
8	C	18	C	28	C	38	D
9	A	19	B	29	B	39	A
10	B	20	B	30	A	40	B

Detailed Explanations

1. Correct answer: B

A declaration statement must end with a semicolon. Omitting it violates C syntax, so the compiler reports an error before an executable can be produced; it is not a run-time or link-time problem.

2. Correct answer: C

A C identifier cannot begin with a digit, so 2marks is invalid. The other names begin with a letter or underscore and contain only permitted letters, digits, or underscores.

3. Correct answer: B

`<stdio.h>` supplies the declaration of `printf`. Without that declaration, a modern C compiler must diagnose the call; the preprocessor does not automatically insert the header.

4. Correct answer: C

`gcc check.c -o check` translates the source and requests the output name `check`. The other commands reverse the source/output positions or use unsupported syntax.

5. Correct answer: B

The linker combines object modules and connects referenced library routines to produce an executable. Editing creates source text, while input validation and looping occur within programs.

6. Correct answer: C

`%d` matches an `int`, and `&age` supplies the address where `scanf` must store the input. The other choices use the wrong format, omit the address operator, or have invalid call syntax.

7. Correct answer: B

`==` compares two values for equality. A single `=` performs assignment, while `!=` and `>=` test different relationships.

8. Correct answer: C

The code is syntactically valid and can be translated. When the entered divisor is zero, evaluating `100 / divisor` performs illegal integer division by zero during execution.

9. Correct answer: A

`i++` changes the loop-control variable so that `i <= 5` eventually becomes false. Resetting `i`, using `continue`, or printing another number does not provide the needed progress.

10. Correct answer: B

`break` ends the current `switch` after the selected case has completed. Without it, execution normally continues into the following case statements, which may cause an unintended logical result.

11. Correct answer: B

A prototype declares the function's interface before a call: its name, return type, and parameter types. This enables the compiler to check calls even when the full definition appears later.

12. Correct answer: A

A `do-while` begins with `do`, places the body in a block, and ends with `while (condition);`. The terminating semicolon is part of this loop's required syntax.

13. Correct answer: A

Both operands of `5 / 2` are integers, so C performs integer division and discards the fractional part. The code compiles and assigns 2 to `pieces`.

14. Correct answer: B

`score = 75` assigns 75 rather than comparing. The assigned value is nonzero, which C treats as true, so Pass is printed; `score == 75` is the intended comparison.

15. Correct answer: C

Both strict boundaries must be satisfied simultaneously, requiring `&&`. Using `||` accepts values outside the interval, and mathematical chained comparison syntax does not express the intended C condition.

16. Correct answer: B

When `x` is zero, `x != 0` is false. Short-circuit evaluation stops the `&&` expression at that point, preventing evaluation of the unsafe division.

17. Correct answer: A

Relational and equality operations are evaluated before `&&`, which is evaluated before `||`. Thus `(6 != 7 && 0) || (4 >= 6)` becomes `(1 && 0) || 0`, which is 0.

18. Correct answer: C

The semicolon immediately after `while (i <= 3)` forms an empty loop body. Because `i` is never changed inside that empty loop, the condition remains true and the later block is never reached.

19. Correct answer: B

At `i == 3`, `continue` skips every remaining statement in the current iteration, including `i++`. The next condition check therefore sees 3 again, creating an infinite loop.

20. Correct answer: B

Case 2 prints B but has no `break`, so execution falls through to case 3. Case 3 prints C and then breaks, producing BC; `default` is not reached.

21. Correct answer: C

`%d` requires an `int`, but the argument is a `double`. This mismatch gives undefined behavior, so neither a particular integer nor a floating-point output is guaranteed; a well-configured compiler will normally warn about it.

22. Correct answer: B

For `scanf`, `%lf` matches a `double`, and `&distance` supplies its address. Omitting `&` can lead to invalid memory access, while `%f` in `scanf` is for a `float`.

23. Correct answer: A

The remainder operator `%` is defined for integer operands. Both `a` and `b` are `double`, so the compiler diagnoses invalid operands rather than performing floating-point remainder.

24. Correct answer: A

`<math.h>` declares mathematical library functions such as `sqrt` and `pow`. `#include` directives do not take a trailing semicolon, and loop-control statements cannot replace the missing declaration.

25. Correct answer: A

The prototype promises that `calculate` returns `double`, but the definition declares an `int` return type. These declarations conflict, so the compiler must diagnose them; parameter names themselves do not need to differ.

26. Correct answer: A

C passes the value of `value` into the parameter `n`, creating a separate local copy. Changing `n` therefore does not change the caller's variable, so the program prints 5.

27. Correct answer: B

A C string must end with the null character `\0`. This five-element array contains exactly five visible characters and no terminator, so `%s` may read beyond the array while searching for one.

28. Correct answer: C

`main` calls `first`, which prints A and then calls `second`. `second` prints B, so the combined output is AB; the prototype makes the later function known before the call.

29. Correct answer: B

Starting at 1 with the test `i < 5` produces only 1, 2, 3, and 4. Changing the condition to `i <= 5` includes the required upper endpoint without changing the initialization or update.

30. Correct answer: A

After copying the visible characters, the destination must be terminated with `reversed[length] = '\0'`; . A newline is printable data, while a number or format string does not serve as the required string terminator.

31. Correct answer: B

Without braces, only the first statement following the `for` header belongs to the loop. Therefore, the numbers 0 1 2 are printed during the iterations and Done is printed once afterward.

32. Correct answer: C

`const` makes MAX non-modifiable after initialization. Assigning a new value violates a C constraint and requires a compile-time diagnostic; the statement is not ignored.

33. Correct answer: B

Individual helper functions can be translated into object code, but a complete C program requires its entry point. When the executable is built, the linker normally reports the missing `main` rather than selecting another function automatically.

34. Correct answer: C

The trailing semicolon creates an empty loop. The updates change `(i, j)` from `(1, 9)` to `(3, 7)` and then `(5, 5)`; at that point `i < j` is false, and the following `printf` executes once.

35. Correct answer: A

`d == 0` is true, so short-circuit evaluation of `||` skips `x % d`. The dangerous remainder-by-zero operation is never evaluated, and the program prints Protected.

36. Correct answer: B

`scanf("%lf", &value)` correctly reads a `double`, and `printf("%.2f", value)` correctly displays it with two decimal places. Every other choice has an incorrect format, a missing address for input, an address passed to output, or multiple such errors.

37. Correct answer: B

The function receives a copy of `x`, so adding 5 changes only its local parameter. The program is syntactically valid but contains a logical misunderstanding of pass-by-value and prints 4.

38. Correct answer: D

The outer loop runs three times. On each outer iteration, the inner loop increments `total` twice, giving $3 \times 2 = 6$ total increments.

39. Correct answer: A

The values 3 and 6 are skipped by `continue`. Values 1, 2, 4, and 5 are printed; when `i` becomes 7, `break` ends the loop before 7 can be printed.

40. Correct answer: B

Option B checks `a` before using it in the denominator, calculates the discriminant only for a valid quadratic, and checks `d` before calling `sqrt`. Option A validates too late, option C divides by zero and incorrectly rejects a repeated real root, and option D uses `||`, which does not guarantee both safety conditions.

End of Paper